



Design of lightweight on-chip one-dimensional convolutional neural network accelerators for edge-end chips

Haotian Yu^{a,b}, Daibo Zhang^a, Yaguang Yang^a, Silong Chen^{a,b}, Zhiqiang Li^{a,b,*}

^a Institute of Microelectronics of the Chinese Academy of Sciences, No. 3 Bei-Tu-Cheng West Road, Chaoyang, Beijing, 100029, China

^b University of Chinese Academy of Sciences, Beijing, 100049, China

ARTICLE INFO

Keywords:

1D-CNN
On-chip accelerator
Edge-end chip
Compilation

ABSTRACT

One-dimensional convolutional neural networks (1D-CNNs) play a crucial role in edge computing applications. To address the challenges of deploying 1D-CNNs on edge devices, this paper proposes a lightweight on-chip 1D-CNN accelerator and its corresponding compilation and deployment process for edge-end chips. The proposed accelerator architecture features an innovative two-stage storage structure, a configurable multiplier-adder tree array, and a comparator array, with an integrated CORDIC core to enhance support for various 1D-CNN architectures. The compilation and deployment process supports operator conversion and fusion, increasing circuit utilization and reducing memory access demands. The accelerator was synthesized using the Global Foundries 22 nm process, achieving a peak throughput of 5.8 GOP/s and a power efficiency of 823.4 GOP/s/W at 200 MHz with an area of 0.056 mm². Comparative testing showed that the proposed design's average computation delay and energy efficiency are 0.520 and 0.145 times that of the ARM Cortex-M7, respectively, demonstrating the proposed accelerator's superior performance and energy efficiency.

1. Introduction

Compared to classical 2D convolutional neural networks (2D-CNNs), one-dimensional convolutional neural networks (1D-CNNs) offer several advantages, including lower computational complexity, a more compact structure, and natural suitability for processing one-dimensional data signals [1]. Consequently, 1D-CNNs are extensively employed across various domains, such as arrhythmia detection [2–4], automatic speech recognition [5–7], and structural damage detection [8–10]. Fig. 1 illustrates the schematic diagram of a compact 1D-CNN architecture that is applied for damage detection in bearing structures [11]. In many of these applications, there is a growing demand for deploying algorithms on edge devices. However, the limited computational power of microcontroller units (MCUs) and other edge devices poses significant challenges for directly running 1D-CNN algorithms. While several high-performance CNN accelerators have been proposed [12–16], they are predominantly optimized for neural networks with high computational density, leading to inefficiencies when applied to 1D-CNNs. These inefficiencies result in unnecessary power consumption and increased area, making it difficult to meet the stringent requirements of edge environments. Consequently, the application of these accelerators in edge

scenarios remains limited.

Currently, accelerators for 1D-CNNs can be broadly categorized into those designed for fixed neural network structures and those for general-purpose structures. Xu et al. [17] proposed a 1D-CNN algorithm for speaker recognition, along with an FPGA-based acceleration circuit. This approach significantly reduced computational complexity compared to ResNet20, with only a 1 % reduction in accuracy. Chen et al. [18] presented a VLSI implementation of a 1D-CNN for abnormal heartbeat detection, achieving a detection accuracy of 95.14 % under conditions of 10 kHz frequency, 0.67 mm area, and 3.79 μW power consumption. Vitolo et al. [19] introduced a low-power hardware accelerator for audio PDM-to-PCM conversion based on 1D-CNN, implemented using FPGA, which achieved a 41.56 dB SNR using 606 LUTs and 410 FFs. Liu et al. [20] proposed a 1D-CNN for ECG classification, along with its FPGA implementation, achieving 99.19 % classification accuracy and 12.82 GOPS/kLUT hardware efficiency at 200 MHz. Syu et al. [21] developed a binary 1D-CNN and its FPGA implementation for bearing fault diagnosis, which performed inference with 98.5 % accuracy in 0.42 ms. Additionally, related studies [22,23] have explored the use of 1D-CNN-LSTM for inference and proposed corresponding hardware implementations. However, these accelerators, designed to optimize

* Corresponding author. Institute of Microelectronics of the Chinese Academy of Sciences, No. 3 Bei-Tu-Cheng West Road, Chaoyang, Beijing, 100029, China.

E-mail addresses: yuhaotian@ime.ac.cn (H. Yu), zhangdaibo@ime.ac.cn (D. Zhang), yangyaguang@ime.ac.cn (Y. Yang), chensilong@ime.ac.cn (S. Chen), lizhiqiang@ime.ac.cn (Z. Li).

<https://doi.org/10.1016/j.mejo.2025.106570>

Received 10 September 2024; Received in revised form 6 December 2024; Accepted 15 January 2025

Available online 16 January 2025

1879-2391/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

specific 1D-CNN structures, tend to have relatively fixed computing units and data paths. While they achieve excellent performance for their targeted neural network structures, their architecture restricts them to running only a single or slightly modified neural network structure, limiting their applicability to other neural networks.

Another category of accelerators is the 1D-CNN accelerator designed for general neural network structures. Wang et al. [24] proposed a one-dimensional chain structure for accelerating deep CNNs, utilizing a set of adjacent dual-channel processing engines to form a 1D contraction primitive that accelerates convolution operations. This design achieved a peak throughput of 806.4 GOP/s and a power efficiency of 1421 GOP/s/W. Sanchez et al. [25] introduced a CNN accelerator for edge-end real-time video stream processing, capable of running dynamic one-dimensional convolution on pixel streams. Their FPGA implementation demonstrated a 7.3-fold reduction in dynamic power consumption compared to conventional 2D convolution CNNs, while maintaining 60 frames per second with only a 2 % accuracy loss. Xu et al. [26] proposed a CNN accelerator that can be implemented on low-cost FPGAs. This design unifies the computation of convolutions with different kernel sizes into a single calculation method, reducing the dimensionality of the computation array from 2D to 1D. Testing on the ZYNQ platform yielded FPS/DSP and GOPS/DSP values of 0.55 and 0.35, respectively. Additionally, universal 1D-CNN accelerators have been proposed and applied to fixed neural network structures [27–29]. These designs generally operate neural network using configurable processors, achieving excellent performance in their respective application fields. However, most of these 1D-CNN accelerators are implemented on FPGAs or as full-chip designs. While they can achieve extremely high performance when utilizing the entire chip, they also require significant area and high-speed memory resources, making it challenging to integrate them into edge SoCs or MCUs.

To address the aforementioned challenges, this paper proposes a lightweight one-dimensional convolutional neural network (1D-CNN) accelerator designed for integration into edge device chips. The proposed solution encompasses both the hardware circuit architecture and the software compilation and deployment process. In the hardware architecture, we present a two-level storage structure, a configurable multiplier-adder tree array, and a comparator array, which collectively enhance the accelerator's compatibility with various 1D-CNN architectures. Additionally, a CORDIC core is integrated to improve support for softmax function. On the software side, we present a comprehensive neural network compilation process that includes operator conversion and operator fusion operations. These operations are employed to maximize circuit utilization and minimize memory access demands, thereby enabling efficient deployment of the 1D-CNN on the edge device's on-chip accelerator.

2. Accelerator hardware structure

The structure of the proposed accelerator is illustrated in Fig. 2. The accelerator is interfaced with the on-chip bus, where it is configured by the CPU via the bus and operates as a master to access external memory (alternatively, it can access external memory through a dedicated data path). The accelerator comprises a I/O interface, a multiplier-adder tree array, a comparator array, global memory, register bank, a CORDIC core and a controller.

The accelerator's I/O interface serves dual functions: it receives configuration information from the CPU and sends an interrupt signal back to the CPU upon completion of operations, while also managing data reading and writing between the external memory. The multiplier-adder tree array consists of 15 multipliers and 15 adders, which can be configured into multiplier-adder tree of size 3 or 5 using a multiway selector. Fig. 3 presents a schematic diagram of a portion of the multiplier-adder tree array, where each multiplier is 16 bits and each adder is 32 bits. The final operation result is truncated to 16 bits using a truncator. Convolution operations with kernel sizes less than or equal to 5 can be completed in a single cycle, while those with kernel sizes greater than 5 are completed over multiple cycles using intermediate data registers. For convolution layers with kernel sizes smaller than 15, parallel acceleration across multiple convolution kernels is possible. For layers with kernel sizes greater than or equal to 15, acceleration is achieved for a single convolution kernel computation. The comparator array is composed of 10 comparators and 10 registers, supporting up to 10 parallel comparison operations. This array can be configured to perform ReLU functions as well as maximum pooling operations.

When the mux signal is high, all mux devices route the “1” path open and the “0” path closed, resulting in $\text{Output0} = \text{Data0Weight0} + \text{Data1Weight1} + \text{Data2Weight2} + \text{Bias0}$, with similar outputs for other Output ports. At this time, the multiplier-adder tree array is configured as five filters array of size 3. Conversely, when the mux signal is low, $\text{Output0} = \text{Data0Weight0} + \text{Data1Weight1} + \text{Data2Weight2} + \text{Data3Weight3} + \text{Data4Weight4} + \text{Bias0}$, with similar outputs for Output1 and Output2 ports, while Output3 and Output4 ports produce meaningless outputs, and bias3 and bias4 ports are unused. During this configuration, the multiplier-adder tree array is arranged as three filters array of size 5. Therefore, although the entire multiplier-adder tree array consists of multiple cascaded multipliers and adders, the presence of muxes limits the longest path to approximately the length of one multiplier followed by five cascaded adders. Compared to traditional systolic arrays, this design exhibits a slight decrease in maximum frequency but eliminates multiple high-switching-activity data registers within the systolic arrays, resulting in significant savings in dynamic power consumption. This compromise is a viable consideration for edge devices.

The storage structure of the accelerator is divided into two components: a global memory and a register group. The global memory consists of a relatively large-capacity dual-port SRAM, which includes input

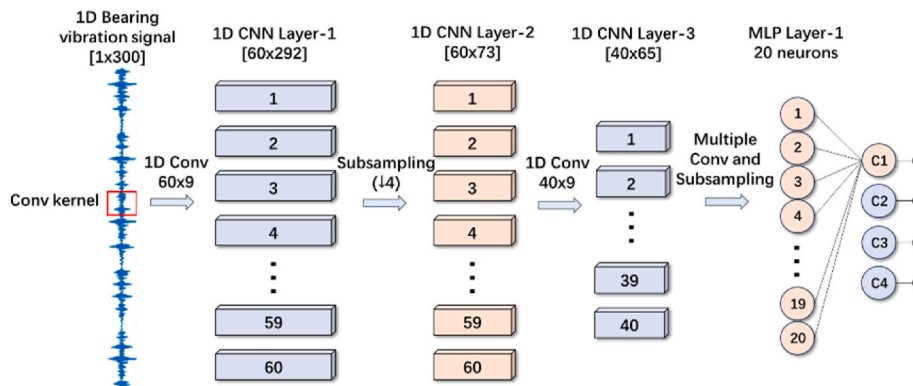


Fig. 1. Schematic diagram of a compact 1D-CNN applied for damage detection in bearing structures.

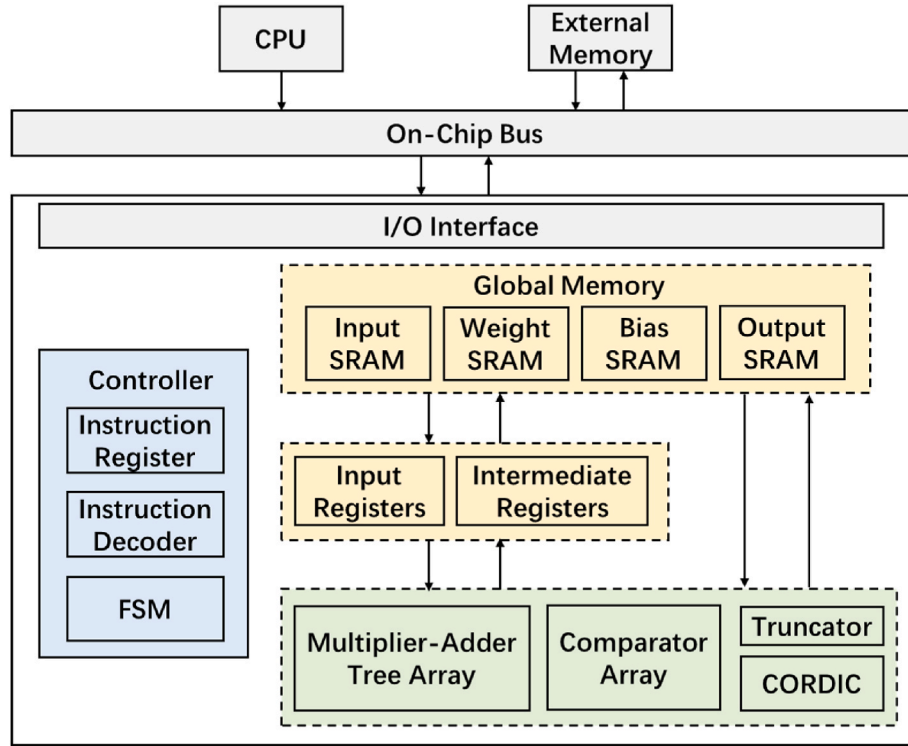


Fig. 2. Overall structure of the proposed accelerator.

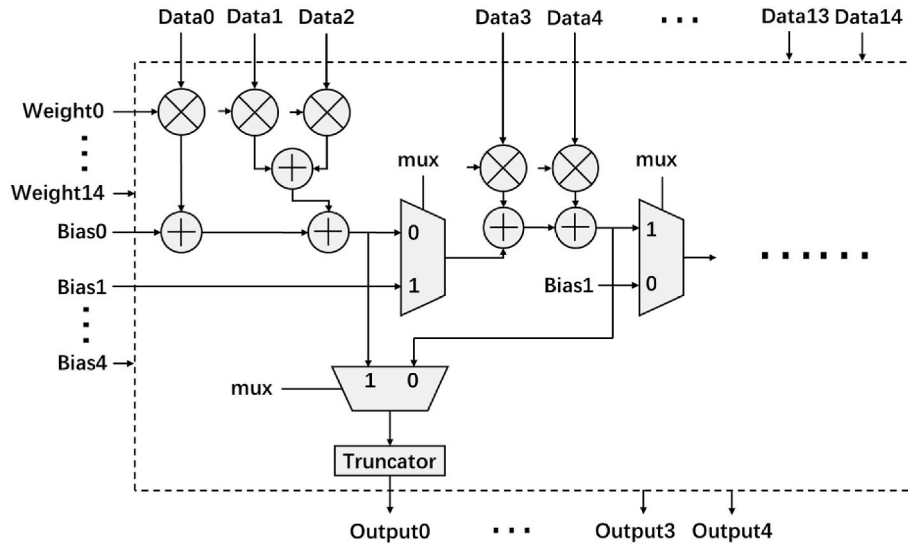


Fig. 3. Schematic of the multiplier-adder tree array, illustrating multiplier and adder operations with filter size and data path control via multiway selectors.

data SRAM, weight SRAM, bias SRAM, and write-back SRAM. The register group is a smaller-capacity, multi-port, randomly accessible set of registers, comprising an input register group and an intermediate data register group. The input data SRAM stores input data and transfers it to the register group in blocks. The input register group can be accessed by the controller state machine through random addresses across multiple ports, allowing data to be output to the multiplier-adder tree array or comparator array. Additionally, the sizes of the SRAM and register groups associated with input and output operations are increased to support ping-pong reading and writing of input and output data. If the external memory is occupied for an extended period, the accelerator halts operations and resumes once access to the external memory is restored.

The CORDIC core supports the final softmax layer in certain neural network structures by performing exponential and division operations, thus enabling the computation of the softmax function in conjunction with the multiplier-adder tree array. The final result is output through a truncator. It is important to note that the CORDIC core has a limited input size range; therefore, incorporating a batch normalization layer is recommended to normalize the data within the network when using the softmax function. The controller state machine manages the entire accelerator by setting appropriate values for the global memory read-write address registers, the read-write path control registers, the register group read-write address registers, and other related registers according to the program instructions. This configuration allows comprehensive control of the accelerator during operation.

Additionally, the intermediate data register group within the accelerator enables the integration of ReLU functions and conventional pooling operations, allowing them to run in a pipelined manner. Fig. 4 illustrates the pipeline structure of convolution, ReLU, and maximum pooling operations between registers.

3. Software design

The software process of the proposed design is illustrated in Fig. 5. Machine learning libraries, such as PyTorch, export the convolutional neural network model to be deployed into TVM [30]. TVM compiles the network model with C code as the target, without focusing on a specific hardware device. The optimized graph structure and its weight parameters are then output to the compiler proposed in this paper.

First, the compiler performs depth-first traversal, operator conversion, and operator fusion on the graph starting from the input node. It then converts each layer operation in the graph into the corresponding opcode for the accelerator, generating the accelerator's running instructions with placeholders. Next, the weight (bias) parameter storage structure is adapted to fit the accelerator and its instructions. The compiler calculates the starting address and length of the weights for each layer and determines the storage address for the calculation results, ensuring minimal memory usage. This information, including addresses and lengths, is then filled into the accelerator running instructions to produce a complete program. In the Software Development Kit (SDK) programming for the main CPU of the embedded device, the instructions and weights are stored at the appropriate memory addresses. During execution, the instructions and weights are loaded into the correct memory locations, and the required memory space is allocated. This memory space comprises program instructions, weights, biases, input data, and intermediate results. The accelerator is then configured and started by writing the instruction address, instruction length, and other relevant information to its address. Once all instructions are completed, the accelerator notifies the CPU of the program's completion and returns the results via an interrupt.

The design supports common hierarchical structures of one-dimensional convolutional neural networks, and the accelerator's instruction set is tailored to the hierarchical nature of these networks, balancing flexibility and simplicity. The instruction types include convolution and pooling instructions, with their structure depicted in Fig. 6. For the convolutional layer, the instruction format supports parameters such as the number of input signal channels, the length and number of convolutional kernels, and the stride length. It also includes a pipeline operation flag to enable direct combination with common maximum and average pooling operations, specifically for pooling kernels of size 3 or 5 with strides equal to the kernel size, immediately following the convolution operation. Maximum pooling is pipelined using the entire comparator array and register groups, while average pooling is performed by leveraging the multiplicative tree array when the relevant section of the intermediate data register group is full. During this operation, the convolution process is momentarily paused for one clock cycle. In this design, batch normalization and linear layers are also transformed into convolution operations that correspond to multiplication weights and sequences, allowing them to be processed accordingly. Additionally, softmax function operations can be directly combined with the linear layer operations.

For the pooling layer, the design supports both average pooling and maximum pooling, with configurable parameters including pooling

type, stride, and kernel size. Regarding activation functions, the design accommodates the widely used ReLU and softmax functions. ReLU is employed as the activation function for conventional convolution operations and is executed immediately after the convolution operation by default. In contrast, the softmax function is only supported for a limited amount of data in the final layer of the network. The CORDIC module performs the exponential and division operations required by the softmax function, while the multiplier-adder tree array handles the addition operations. Although the softmax function's calculations based on the CORDIC algorithm may introduce minor errors, these errors have minimal impact on classification performance [31].

4. Results and discussion

In this design, a 16-bit data width was selected to accommodate various edge signal data types [32]. The dimensions of the input data storage, weight storage, and bias storage are 1024×16 bits, 512×16 bits, and 128×16 bits, respectively, with all storage units generated by a memory compiler to minimize area utilization. The synthesis results of the proposed design, based on the Global Foundries 22 nm process using Synopsys Design Compiler, are presented in Table 1.

The approach in Ref. [33] achieved high power efficiency through sparse computation and low frequency, while [27] employed singular value decomposition to compress weights, resulting in high throughput. In contrast [24], achieved high power efficiency but at the cost of a large area. The design proposed in this paper provides high power efficiency and area efficiency, with values of 823.4 GOP/s/W and 103.6 GOP/s/mm², respectively, within a compact area. Additionally, it offers enhanced configurability for 1D-CNNs. The reduction in area is primarily achieved by minimizing the amount of SRAM, which imposes a limit on the size of the 1D-CNNs that can be supported. To accommodate larger 1D-CNNs, increasing the SRAM capacity would be necessary. Although the multiplier-adder tree array employed in this design comprises only 15 multipliers and adders, which limits its peak performance when processing convolutional layers with kernel sizes larger than 15, resulting in lower throughput, several considerations mitigate this limitation. Edge devices are characterized by lower memory chip frequencies and constrained on-chip and off-chip communication bandwidths. Furthermore, the computational complexity of 1D-CNNs is relatively low. The proposed design reduces the occupation of on-chip communication bandwidth and minimizes the probability of blocking other on-chip devices' access to memory due to the accelerator's utilization of memory bandwidth. Despite these constraints, the design still meets the computational demands of most 1D-CNNs and achieves high circuit utilization when processing small-sized 1D-CNNs. Additionally, compared to the literature discussed, this design significantly enhances pooling layer configurability, supporting network structures with larger pooling sizes and lower computational speeds than those of convolutional layers. Additionally, by integrating a CORDIC computing core, the proposed accelerator can support softmax functions, thereby broadening the range of neural network structures it can efficiently handle.

The ARM Cortex-M7 core is a widely used high-performance processor core in embedded devices. To evaluate the proposed design, we conducted software simulation tests comparing our accelerator with the ARM Cortex-M7 core in terms of computational delay and energy efficiency. In these tests, the Cortex-M7 operated at 1 GHz, while the proposed design operated at 200 MHz, with off-chip memory configured as a DDR3 chip. Lightweight 1D-CNN models, typical of various edge



Fig. 4. Pipeline structure of convolution, ReLU, and maximum pooling operations; the sequence numbers indicate data flow order.

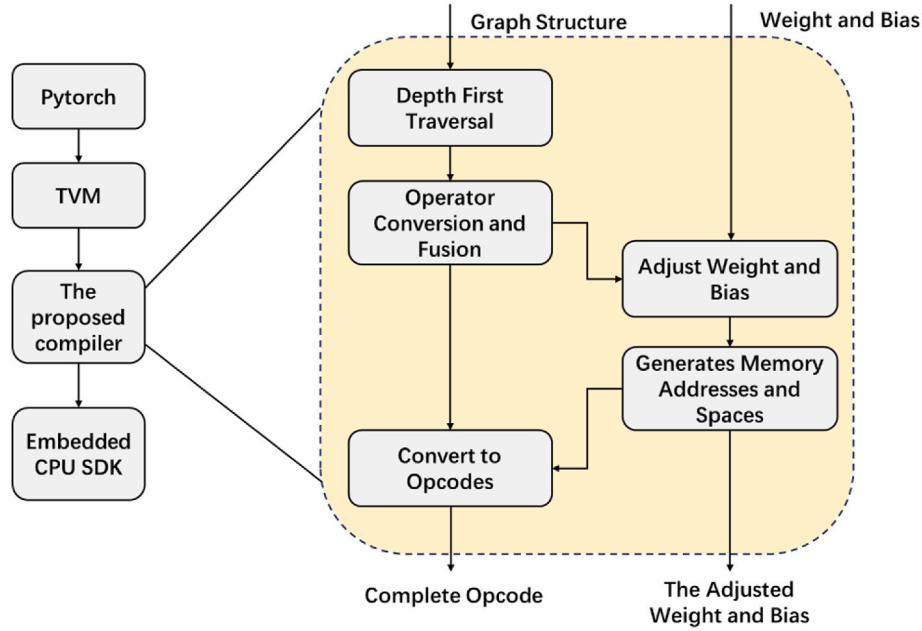


Fig. 5. Flowchart of the compilation and deployment process.

Convolution Instruction

Operation Type	Data Address	Weight Address	Bias Address	Write Back Address	Input Channel	Output Channel	Kernel Size	Stride	Pipeline Operation	Softmax
0	[23:0]	[23:0]	[23:0]	[23:0]	[6:0]	[6:0]	[8:0]	[2:0]	[2:0]	[0:0]

Pooling Instruction

Operation Type	Data Address	Write Back Address	Pooling Type	Kernel Size	Stride
1	[23:0]	[23:0]	[1:0]	[8:0]	[2:0]

Fig. 6. Instruction structure of the accelerator.

Table 1

Circuit performance metrics of the proposed accelerator.

Performance Parameters	[33]	[27]	[24]	This work
Process (nm)	40	65	28	22
Voltage (V)	1.1	1.08	0.9	0.45
Frequency (MHz)	2	400	700	200
SRAM (KB)	48	12.7	352	3.4
Area (mm ²)	2.044	1.016	3715k gates	0.056
Power (mW)	0.841	46.5	567.5	7.044
Throughput (GOP/s)	0.228	1400	806.4	5.8
Power Efficiency (GOP/s/W)	271	30.11	1421	823.4

application scenarios, was used for testing. The power consumption data for the Cortex-M7 is utilized from the official documentation [34], which provides figures based on a 16 nm process technology. The final comparison results are presented in Table 2. The test results, obtained from running 1D-CNN models across multiple application scenarios and structures, demonstrated that the average computational delay of the proposed design was 0.520 times that of the Cortex-M7, the average energy efficiency of the proposed design was 0.145 times that of the Cortex-M7. Additionally, the additional accuracy loss incurred is relatively minor. These results indicate that the proposed design offers significantly higher computational efficiency and lower energy consumption for 1D-CNN applications compared to the Cortex-M7.

Table 2

Software performance metrics of the proposed accelerator.

Performance Parameters	[35]	[36]	[37]	[38]	[39]
MACs	68.9M	18.8M	5.8M	1.8M	0.5M
Parameters	62.9 K	2.3M	121.6 K	94.5 K	171.5 K
Layer numbers	8	9	14	10	9
Test device	M7	This work	M7	This work	M7
Calculation delay (ms)	57.4	29.3	15.6	8.2	4.8
Energy efficiency (uj/classification)	1,447.5	206.1	394.1	57.7	120.8
Loss of accuracy	0.23 %	0.48 %	0.52 %	0.79 %	0.43 %
					0.57 %
					0.17 %
					0.24 %
					0.22 %
					0.35 %

5. Conclusions

This paper presents a lightweight on-chip one-dimensional convolutional neural network (1D-CNN) accelerator designed for edge-end chips, consisting of two key components: the accelerator circuit architecture and the compiler deployment process. The proposed accelerator enables the efficient deployment of neural networks on edge devices. We synthesized the circuit using the Global Foundries 22 nm process to validate the performance of the proposed accelerator, demonstrating suitable power efficiency in a compact area. Lightweight 1D-CNN models were deployed across various edge application scenarios on both the Cortex-M7 core and the proposed accelerator for comparative testing. The results demonstrate that the proposed accelerator achieves higher computational efficiency and lower energy consumption compared to the Cortex-M7 core.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This research was funded by the project “CAS Youth Interdisciplinary Team”, grant no. (JCTD-2022-07). All authors have read and agreed to publish this paper.

Data availability

The authors do not have permission to share data.

References

- [1] S. Kiranyaz, O. Avci, O. Abdeljaber, et al., 1D convolutional neural networks and applications: a survey, *Mech. Syst. Signal Process.* (2021) 151.
- [2] U.R. Acharya, H. Fujita, O.S. Lih, et al., Automated detection of arrhythmias using different intervals of tachycardia ECG segments with convolutional neural network, *Inf. Sci.* 405 (2017) 81–90.
- [3] Z. Xiong, M.K. Stiles, J. Zhao, et al., Robust ECG signal classification for detection of atrial fibrillation using a novel neural network, in: *Proceedings of the 44th Computing in Cardiology Conference (CinC)*, FRANCE, Rennes, 2017. F 2017 Sep 24–27, 2017 [C].
- [4] U.R. Acharya, S.L. Oh, Y. Hagiwara, et al., A deep convolutional neural network model to classify heartbeats, *Comput. Biol. Med.* 89 (2017) 389–396.
- [5] M. Bi, Y. Qian, K. Yu, et al., Very deep convolutional neural networks for LVCSR, in: *Proceedings of the 16th Annual Conference of the International-Speech-Communication-Association (INTERSPEECH 2015)*, Dresden, GERMANY, 2015. F 2015 Sep 06–10, 2015 [C].
- [6] T. Seru, C. Puhrsch, B. Kingsbury, et al., Very deep multilingual convolutional neural networks for LVCSR, in: *Proceedings of the 41st IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2016. Shanghai, PEOPLES R CHINA, F 2016 Mar 20–25, 2016 [C].
- [7] D. Yu, W. Xiong, J. Droppo, et al., Deep convolutional neural networks with layer-wise context expansion and attention, in: *Proceedings of the 17th Annual Conference of the International-Speech-Communication-Association (INTERSPEECH 2016)*, 2016. San Francisco, CA, F 2016 Sep 08–12, 2016 [C].
- [8] O. Abdeljaber, O. Avci, S. Kiranyaz, et al., Real-time vibration-based structural damage detection using one-dimensional convolutional neural networks, *J. Sound Vib.* 388 (2017) 154–170.
- [9] O. Abdeljaber, O. Avci, M.S. Kiranyaz, et al., 1-D CNNs for structural damage detection: verification on a structural health monitoring benchmark data, *Neurocomputing* 275 (2018) 1308–1317.
- [10] O. Avci, O. Abdeljaber, S. Kiranyaz, et al., Wireless and real-time structural damage detection: a novel decentralized method for wireless sensor networks, *J. Sound Vib.* 424 (2018) 158–172.
- [11] L. Eren, T. Ince, S. Kiranyaz, A generic intelligent bearing fault diagnosis system using compact adaptive 1D CNN classifier, *J. Signal Process. Syst. Signal Image Video Technol.* 91 (2) (2019) 179–189.
- [12] N.P. Jouppi, C. Young, N. Patil, et al., In-datacenter performance analysis of a tensor processing unit, in: *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*, CANADA, Toronto, 2017. F 2017 Jun 24–28, 2017 [C].
- [13] V. Sze, Y.-H. Chen, T.-J. Yang, et al., Efficient processing of deep neural networks: a tutorial and survey, *Proc. IEEE* 105 (12) (2017) 2295–2329.
- [14] Y.-H. Chen, T. Krishna, J.S. Emer, et al., Eyeriss: an energy-efficient reconfigurable accelerator for deep convolutional neural networks, *IEEE J. Solid State Circ.* 52 (1) (2017) 127–138.
- [15] P. Chi, S. Li, C. Xu, et al., PRIME: a novel processing-in-memory architecture for neural network computation in ReRAM-based main memory, in: *Proceedings of the 43rd ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*, 2016. Seoul, SOUTH KOREA, F 2016 Jun 18–22, 2016 [C].
- [16] T. Chen, Z. Du, N. Sun, et al., DianNao: a small-footprint high-throughput accelerator for ubiquitous machine-learning, *ACM Sigplan Not.* 49 (4) (2014) 269–283.
- [17] J. Xu, S. Li, J. Jiang, et al., A simplified speaker recognition system based on FPGA platform, *IEEE Access* 8 (2020) 1507–1516.
- [18] Y.H. Chen, Y. Juan, Very-large-scale integration implementation of a convolutional neural network accelerator for abnormal heartbeat detection, *Electron. Lett.* 56 (7) (2020) 330.
- [19] P. Vitolo, G.D. Licciardo, A.C. Amendola, et al., Quantized 1D-CNN for a low-power PDM-to-PCM conversion in TinyML KWS applications, in: *Proceedings of the IEEE International Conference on Artificial Intelligence Circuits and Systems (AICAS) - Intelligent Technology in the Post-Pandemic Era*, Incheon, 2022. SOUTH KOREA, F 2022 Jun 13–15, 2022 [C].
- [20] D. Liu, L. Wei, X. Zou, et al., Design of hybrid multimode convolutional neural NetworkAccelerator for electrocardiogram detection br, *J. Electron. Inf. Technol.* 45 (1) (2023) 33–41.
- [21] Z.-S. Syu, C.-H. Lee, One-dimensional binary convolutional neural network accelerator design for bearing fault diagnosis, *IEEE Sensor. J.* 24 (3) (2024) 3649–3658.
- [22] B. Wu, X. Wu, P. Li, et al., Efficient FPGA implementation of convolutional neural networks and long short-term memory for radar emitter signal recognition, *Sensors* 24 (3) (2024).
- [23] J. Zhou, D. Li, H. Su, Particle accelerator power system early fault diagnosis based on deep learning and multi-sensor feature fusion, *Eng. Res. Express* 6 (2) (2024).
- [24] S. Wang, D. Zhou, X. Han, et al., Chain-NN: an energy-efficient 1D chain architecture for accelerating deep convolutional neural networks; proceedings of the 20th conference and exhibition on design, in: *Automation and Test in Europe (DATE)*, EPFL Campus, Lausanne, SWITZERLAND, 2017. F 2017 Mar 27–31, 2017 [C].
- [25] J. Sanchez, N. Soltani, R. Chamathi, et al., A novel 1D-convolution accelerator for low-power real-time CNN processing on the edge, in: *Proceedings of the IEEE High Performance Extreme Computing Conference (HPEC)*, 2018. Waltham, MA, F 2018 Sep 25–27, 2018 [C].
- [26] Y. Xu, S. Wang, N. Li, et al., Design and implementation of an efficient CNN accelerator for low- cost FPGAs, *IEICE Electron. Express* (2022).
- [27] B. Zhang, J. Han, Z. Huang, et al., A real-time and hardware-efficient processor for skeleton-based action recognition with lightweight convolutional neural network, *IEEE Trans. Circuits Syst. II-Express Briefs* 66 (12) (2019) 2052–2056.
- [28] J. Lu, D. Liu, Z. Liu, et al., Efficient hardware architecture of convolutional neural network for ECG classification in wearable healthcare device, *IEEE Trans. Circuits Syst. I-Regul. Papers* 68 (7) (2021) 2976–2985.
- [29] L. Hui, S. Cao, Z. Chen, et al., Configurable CNN accelerator in speech processing based on vector convolution, in: *Proceedings of the IEEE International Conference on Artificial Intelligence Circuits and Systems (AICAS) - Intelligent Technology in the Post-Pandemic Era*, Incheon, SOUTH KOREA, 2022. F 2022 Jun 13–15, 2022 [C].
- [30] T. Chen, T. Moreau, Z. Jiang, et al., TVM: an automated end-to-end optimizing compiler for deep learning, in: *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2018. Carlsbad, CA, F 2018 Oct 08–10, 2018 [C].
- [31] Y.X. Cao, W.A. Xiao, J.D. Jia, et al., Cordic-based Softmax Acceleration Method of Convolution Neural Network on FPGA; Proceedings of the IEEE International Conference on Artificial Intelligence and Information Systems (ICAIS), PEOPLES R CHINA, Dalian, 2020. F Mar 20–22, 2020 [C].
- [32] <https://physionet.org/>.
- [33] J. Lu, D. Liu, X. Cheng, et al., An efficient unstructured sparse convolutional neural network accelerator for wearable ECG classification device, *IEEE Trans. Circuits Syst. I-Regul. Papers* 69 (11) (2022) 4572–4582.
- [34] <https://developer.arm.com/Processors/Cortex-M7>.
- [35] A. Mahajan, K. Somaraj, M. Sameer, et al., Adopting artificial intelligence powered ConvNet to detect epileptic seizures, in: *Proceedings of the IEEE-EMBS Conference on Biomedical Engineering and Sciences (IECBES) - Leading Modern Healthcare Technology Enhancing Wellness*, Electr Network, 2021. F 2021 Mar 01–03, 2021 [C].
- [36] V.K. Stephen, M. Virutachalam, A.R. Manalang, M.T. Shaikh, An IoT based heart healthcare platform for the Sultanate of Oman, in: *2022 Mohammad Ali Jinnah University International Conference on Computing (MAJICC)*, 2022, pp. 1–7, <https://doi.org/10.1109/MAJICC56935.2022.9994183>. Karachi, Pakistan.
- [37] R. Jana, I. Mukherjee, Efficient seizure prediction and EEG channel selection based on multi-objective optimization, *IEEE Access* 11 (2023) 54112–54121.
- [38] M.R.H. Apu, F. Akter, M.F.A. Lubna, T. Helaly, T.S. Pias, ECG arrhythmia classification using 1D CNN leveraging the resampling technique and Gaussian mixture model, in: *2021 Joint 10th International Conference on Informatics, Electronics & Vision (ICIEV) and 2021 5th International Conference on Imaging, Vision & Pattern Recognition (icIVPR)*, Kitakyushu, Japan, 2021, pp. 1–8, <https://doi.org/10.1109/ICIEVicIVPR52578.2021.9564201>.
- [39] X. Wang, T. Ristanemi, F. Cong, et al., One and two dimensional convolutional neural networks for seizure detection using EEG signals; proceedings of the 28th

European signal processing conference (EUSIPCO), Electron. Netw. (2021). F 2021 Jan 18-22, 2021 [C].